

Area Plots, Histograms, and Bar Charts

What I'm Doing

In this lab, I'm learning how to create and customize three essential types of data visualizations - area plots, histograms, and bar charts - using Python's Matplotlib library.

1. Setting Up

First, I import the core libraries I'll need: NumPy for numerical operations, pandas for data manipulation, and Matplotlib for plotting.

```
import numpy as np
import pandas as pd
import matplotlib as mpl
import matplotlib.pyplot as plt

%matplotlib inline # ensures plots render directly in the notebook
mpl.style.use('ggplot') # for a clean, modern look
print('Matplotlib version: ', mpl.__version__)
```

Matplotlib version: 3.10.9

2. Getting to Know the Dataset

The dataset contains immigration figures to Canada from various countries, spanning the years 1980 to 2013.

```
df_can = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DV0101EN-SkillsNetwork/Data%20Files/Canada.csv')
```

Let's peek at the first few rows to understand the structure:

```
df_can.head()
```

	Country	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	...	2005	:
0	Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	...	3436	
1	Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	...	1223	
2	Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	...	3626	
3	American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	...	0	
4	Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	...	0	

5 rows × 39 columns

```
print(df_can.shape)
```

```
(195, 39)
```

So we have **195 countries** (rows) and **39 columns** (country info + yearly immigration data from 1980 to 2013 plus a `Total` column).

3. Cleaning the Data

The dataset is already fairly clean, but there's one key transformation: I'll **set the Country column as the index**. This will make it easy to look up specific countries using the `.loc` accessor.

```
df_can.set_index('Country', inplace=True)
df_can.head()
```

	Continent	Region	DevName	1980	1981	1982	1983	1984	1985	1986	...	2013	Total
Country													
Afghanistan	Asia	Southern Asia	Developing regions	16	39	39	47	71	340	496	...	3436	
Albania	Europe	Southern Europe	Developed regions	1	0	0	0	0	0	1	...	1223	
Algeria	Africa	Northern Africa	Developing regions	80	67	71	69	63	44	69	...	3626	
American Samoa	Oceania	Polynesia	Developing regions	0	1	0	0	0	0	0	...	0	
Andorra	Europe	Southern Europe	Developed regions	0	0	0	0	0	0	2	...	0	

5 rows × 38 columns

Now the country names serve as row indices. Let's confirm the dimensions and create a handy list of year strings:

```
print('data dimensions:', df_can.shape)

years = list(map(str, range(1980, 2014)))
```

```
data dimensions: (195, 38)
```

This `years` list will come in handy for slicing columns when plotting.

4. Area Plots

Area plots (or **stacked line plots**) are great for visualizing how multiple categories contribute to a total over time. Let's start by identifying the **top 5 countries that contributed the most immigrants** to Canada.

```
df_can.sort_values(['Total'], ascending=False, axis=0, inplace=True)

df_top5 = df_can.head() # get the top 5 entries

df_top5 = df_top5[years].transpose()

df_top5.head()
```

Country	India	China	United Kingdom of Great Britain and Northern Ireland	Philippines	Pakistan
1980	8880	5123	22045	6051	978
1981	8670	6682	24796	5921	972
1982	8147	3308	20620	5249	1201
1983	7338	1863	10015	4562	900
1984	5704	1527	10170	3801	668

After transposing, each column is a country and the index is the year. Let me also convert the index to integers for cleaner plotting:

```
df_top5.index = df_top5.index.map(int)
```

Now I'll create an **unstacked area** plot:

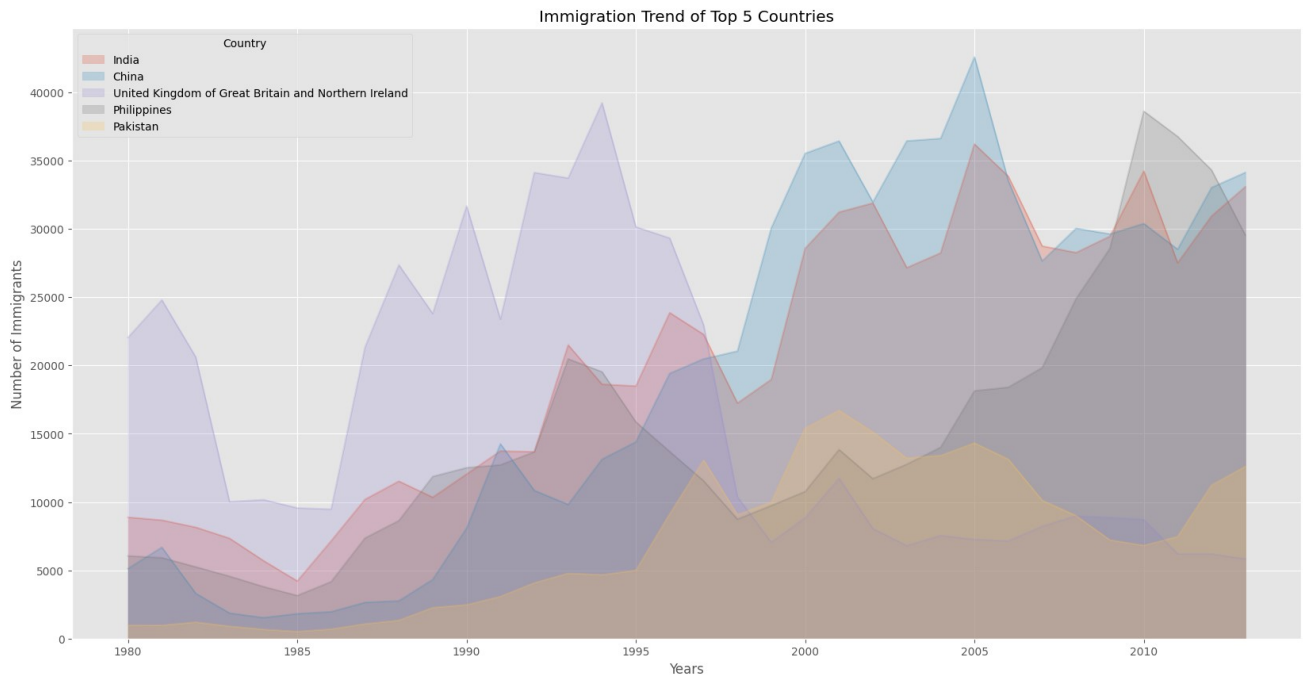
Area plots are **stacked by default**. And to produce a stacked area plot, each column must be either all positive or all negative values (any `NaN`, i.e. not a number, values will default to 0). **To produce an unstacked plot**, set parameter `stacked` to **value False**.

Two Plotting Styles

Method 1 (Scripting layer / procedural) - using `plt.*` functions:

```
df_top5.plot(kind='area',
             stacked=False,
             figsize=(20, 10))

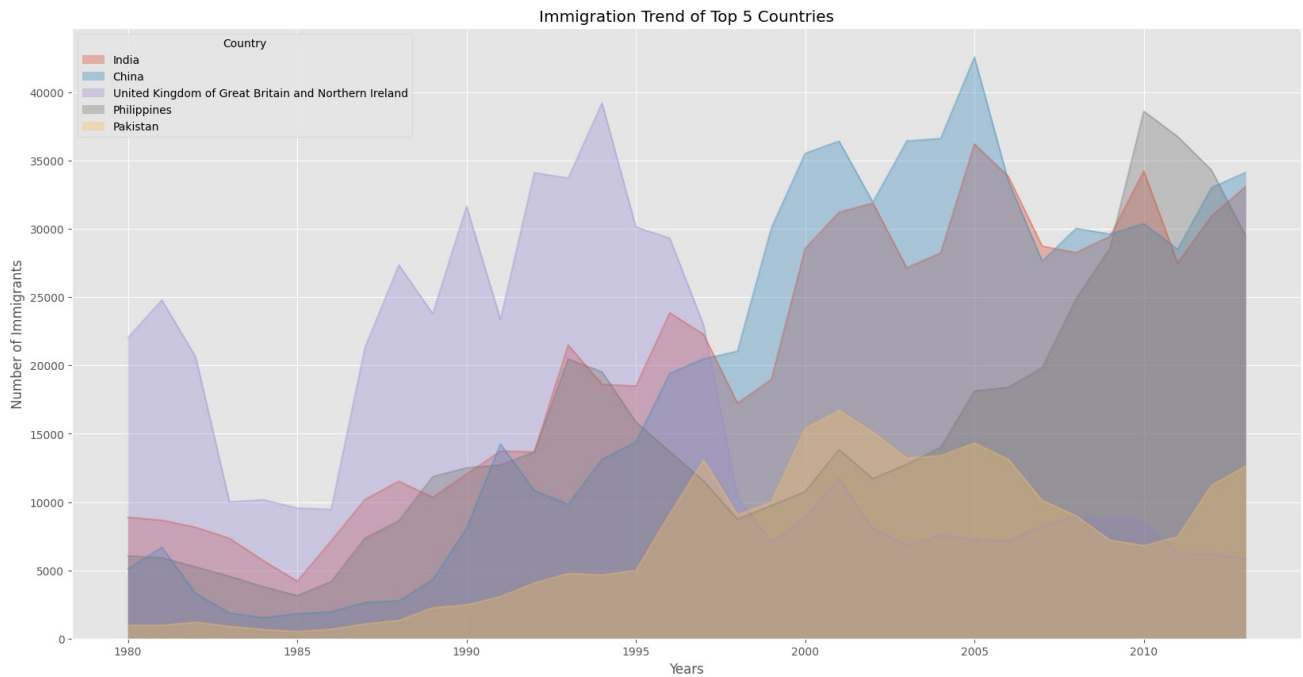
plt.title('Immigration Trend of Top 5 Countries')
plt.ylabel('Number of Immigrants')
plt.xlabel('Years')
plt.show()
```



Result: An area plot with five overlapping shaded regions (one per country), showing immigration trends from 1980 to 2013.

Method 2 (Artist layer / object-oriented) - using an `Axes` instance. This is often preferred for more control.

```
ax = df_top5.plot(kind='area', alpha=0.35, stacked=False, figsize=(20, 10))
ax.set_title('Immigration Trend of Top 5 Countries')
ax.set_ylabel('Number of Immigrants')
ax.set_xlabel('Years'); # semicolon to suppress "Text(0.5, 0, 'Years')" Text
```



Result: Both produce identical plots - the difference is in how you modify plot elements.

Exercise: Least Contributing Countries - Stacked Area Plot (Scripting Layer)

Task: Use the **scripting layer** to create a **stacked** area plot of the **5 countries that contributed the least** to immigration to Canada from 1980 to 2013, with a transparency of 0.45.

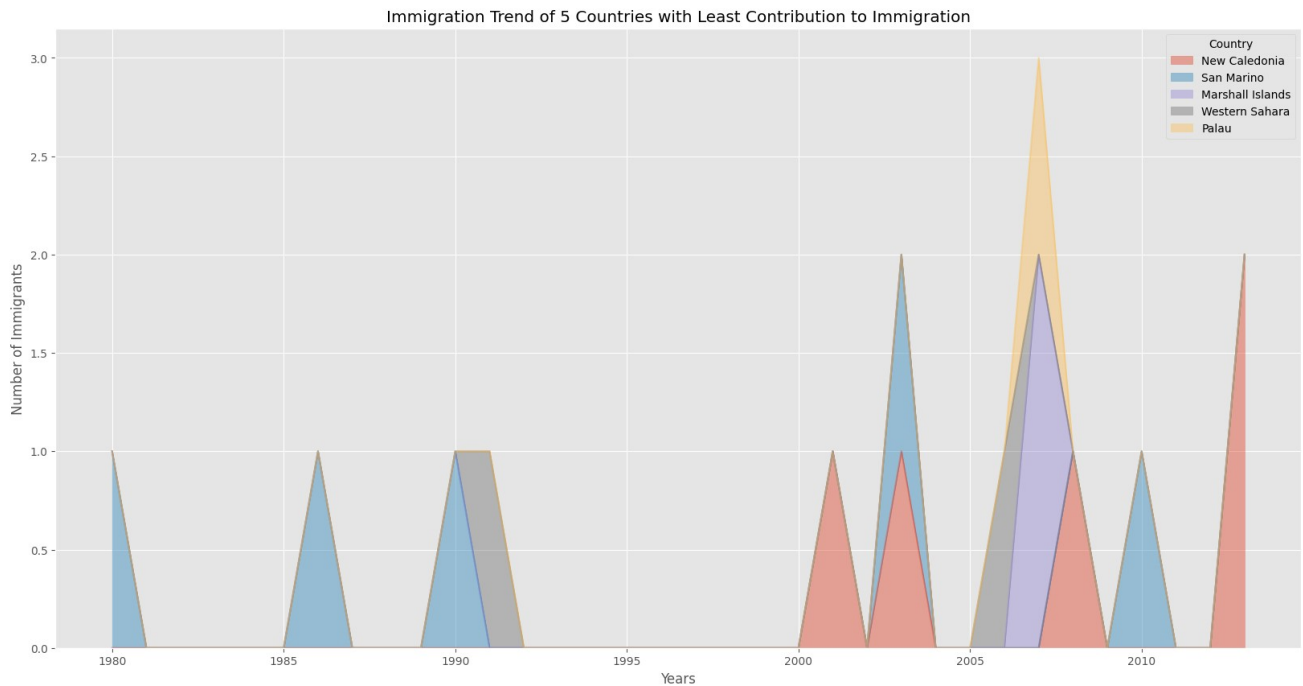
```
df_least5 = df_can.tail(5)

df_least5 = df_least5[years].transpose()

df_least5.index = df_least5.index.map(int)

df_least5.plot(kind='area', alpha=0.45, figsize=(20, 10))

plt.title('Immigration Trend of 5 Countries with Least Contribution to Immigration')
plt.ylabel('Number of Immigrants')
plt.xlabel('Years')
plt.show()
```



Result: Immigration from these countries is not a smooth upward trend but rather event-driven, with sharp spikes in specific years

Exercise: Least Contributing Countries - Unstacked Area Plot (**Artist Layer**)

Task: Use the **artist layer** to create an **unstacked** area plot of the **5 least-contributing countries**, with a transparency of 0.55.

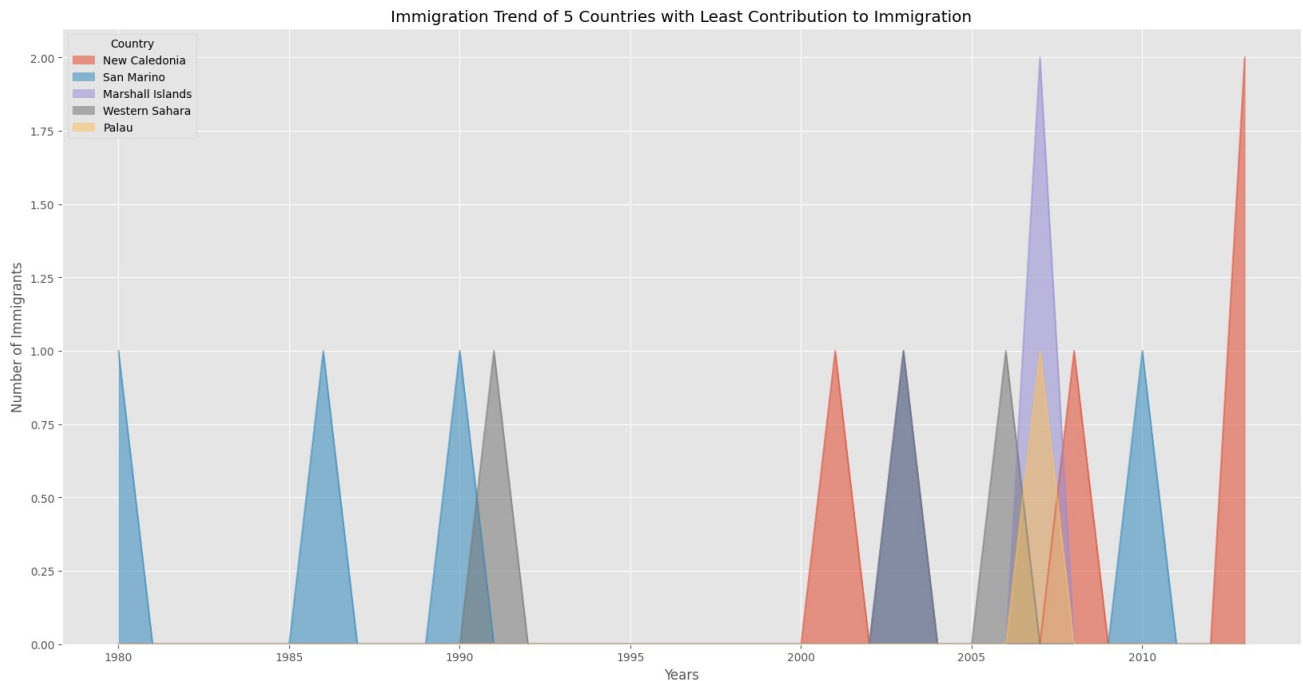
```
df_least5 = df_can.tail(5)

df_least5 = df_least5[years].transpose()

df_least5.index = df_least5.index.map(int)

ax = df_least5.plot(kind='area', alpha=0.55, stacked=False, figsize=(20, 10))

ax.set_title('Immigration Trend of 5 Countries with Least Contribution to Immigration')
ax.set_ylabel('Number of Immigrants')
ax.set_xlabel('Years')
plt.show()
```



The scripting layer (`plt.*`) is simpler for quick plots; the artist layer (`ax.*`) scales better for complex, multi-plot layouts.

5. Histograms

Histograms **show the frequency distribution of a numeric variable** - they group data into bins along the x-axis and count how many observations fall into each bin.

Let's examine the distribution of immigration counts across all countries for the year 2013.

```
df_can['2013'].head()
```

```
Country
India                33087
China                34129
United Kingdom of Great Britain and Northern Ireland    5827
Philippines          29544
Pakistan             12603
Name: 2013, dtype: int64
```

I can use NumPy's `histogram` function to see how the data gets split into bins:

```
count, bin_edges = np.histogram(df_can['2013'])
print(count) # frequency count
print(bin_edges) # bin ranges, default = 10 bins
```

```
[178  11   1   2   0   0   0   0   1   2]
[    0.   3412.9  6825.8 10238.7 13651.6 17064.5 20477.4 23890.3 27303.2
 30716.1 34129. ]
```

So in 2013:

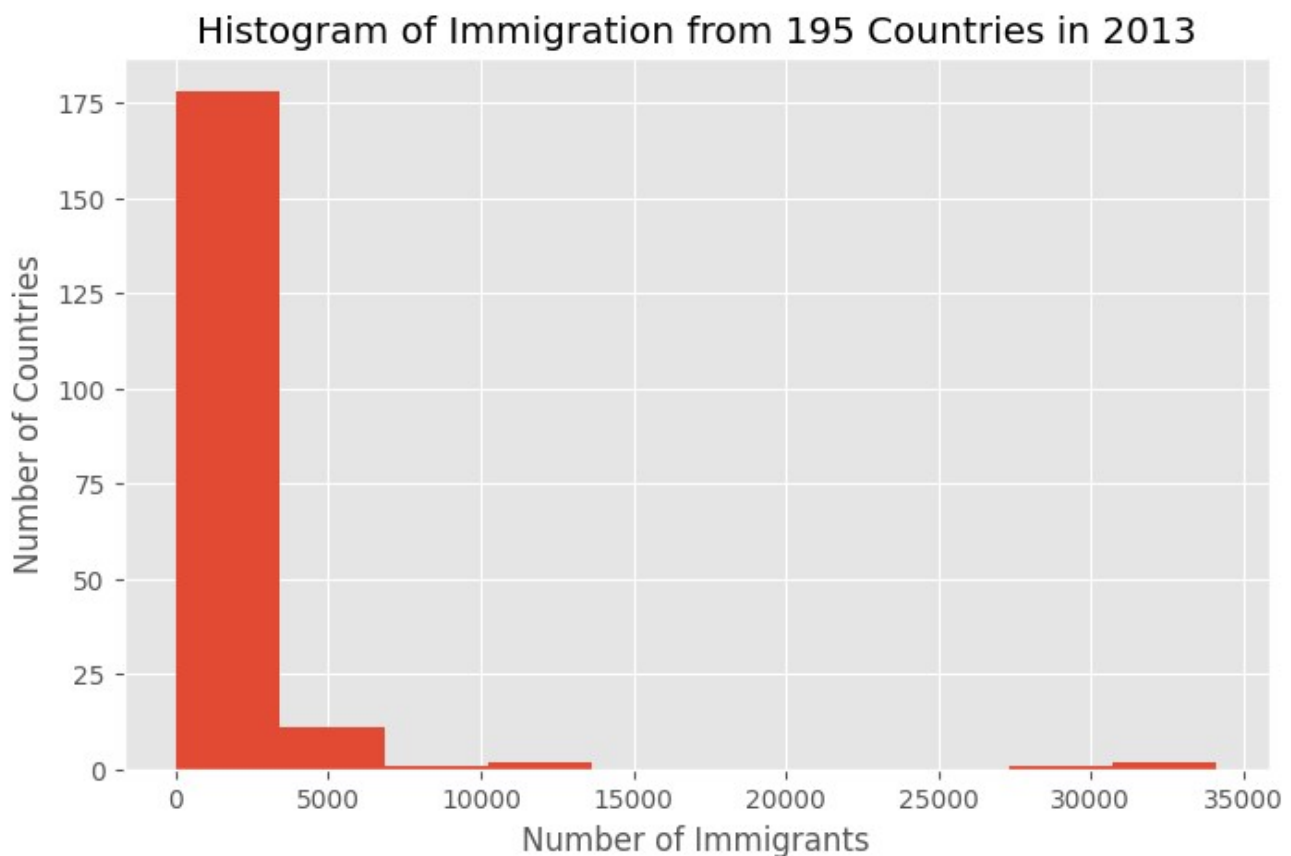
- **178 countries** contributed between 0 and ~3,413 immigrants
- **11 countries** contributed between ~3,413 and ~6,826 immigrants

- And so on — most countries send relatively few immigrants, while a handful send many.

Let's plot this as a histogram:

```
df_can['2013'].plot(kind='hist', figsize=(8, 5)) # kind='hist'

plt.title('Histogram of Immigration from 195 Countries in 2013')
plt.ylabel('Number of Countries')
plt.xlabel('Number of Immigrants')
plt.show()
```



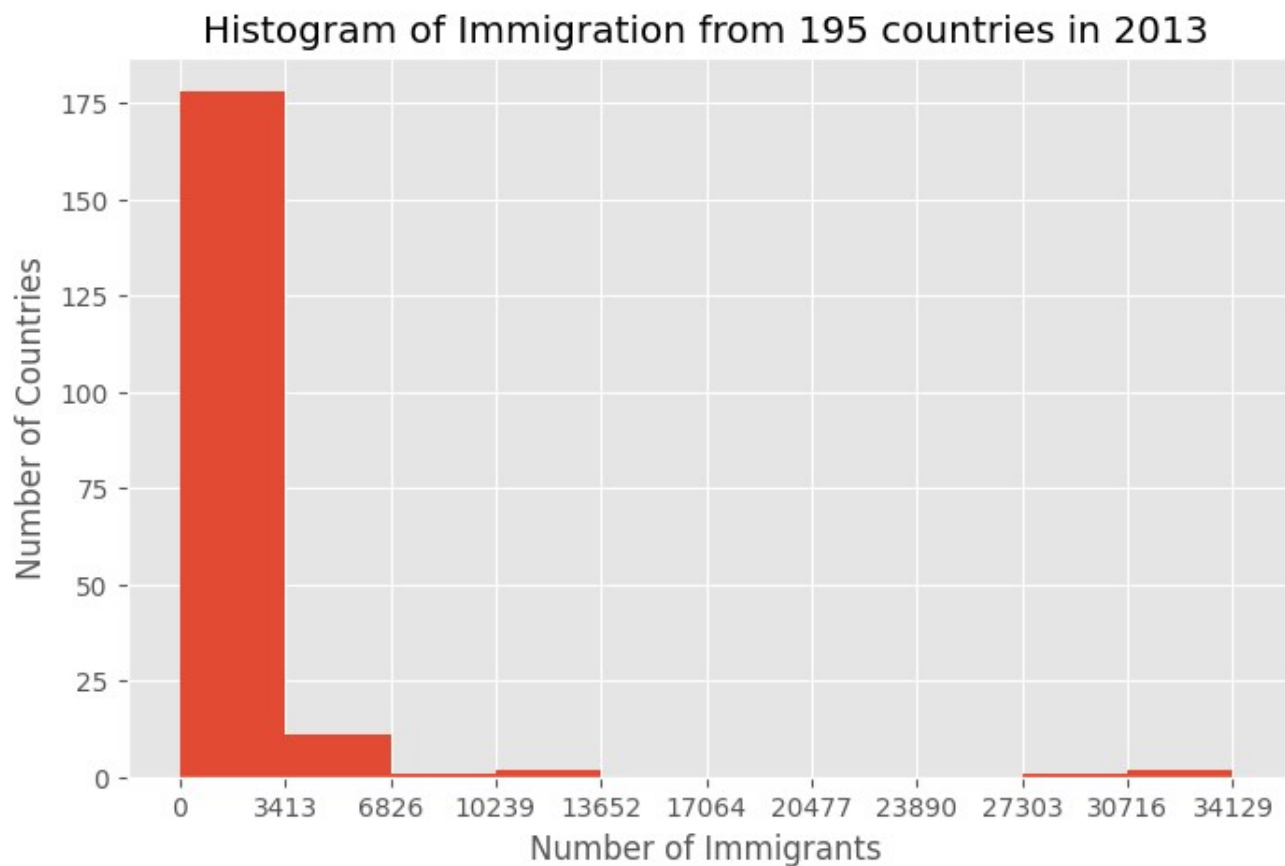
Result: A histogram with a **heavily right-skewed** distribution - most countries cluster in the first bin (low immigration), with a **long tail** of a few high-immigration countries.

The x-axis tick labels don't match the bin edges by default. Let's fix that:

```
count, bin_edges = np.histogram(df_can['2013']) # xticks=bin_edges

df_can['2013'].plot(kind='hist', figsize=(8, 5), xticks=bin_edges)

plt.title('Histogram of Immigration from 195 countries in 2013')
plt.ylabel('Number of Countries')
plt.xlabel('Number of Immigrants')
plt.show()
```

Result: Same histogram, but now the x-axis ticks line up with the actual bin boundaries, making it easier to read the ranges.

Side Note: `df_can['2013'].plot.hist()` is equivalent to `df_can.plot(kind='hist', ...)`. Can be used interchangeably.

Exercise: Immigration Distribution for Denmark, Norway, and Sweden

Task: What is the immigration distribution for Denmark, Norway, and Sweden from 1980 to 2013?

First, let's look at the data:

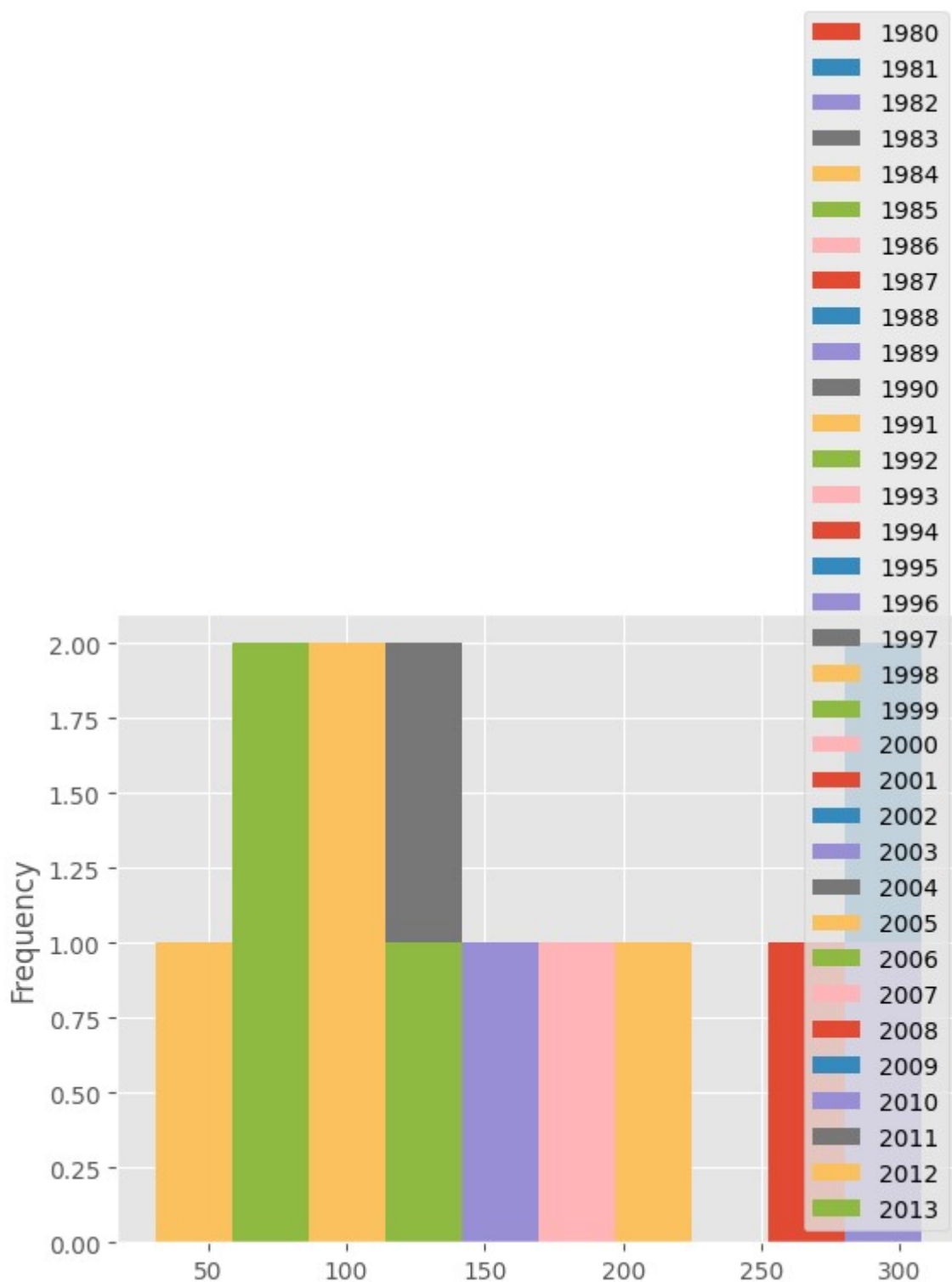
```
df_can.loc[['Denmark', 'Norway', 'Sweden'], years]
```

	1980	1981	1982	1983	1984	1985	1986	1987	1988	1989	...	2004	2005	2006
Country														
Denmark	272	293	299	106	93	73	93	109	129	129	...	89	62	101
Norway	116	77	106	51	31	54	56	80	73	76	...	73	57	53
Sweden	281	308	222	176	128	158	187	198	171	182	...	129	205	139

3 rows × 34 columns

If I plot this directly:

```
df_can.loc[['Denmark', 'Norway', 'Sweden'], years].plot.hist();
```



The result is wrong - pandas plots the frequency distribution of *years*, not countries. The fix is to **transpose** the data so that countries become columns and years become rows, then plot.

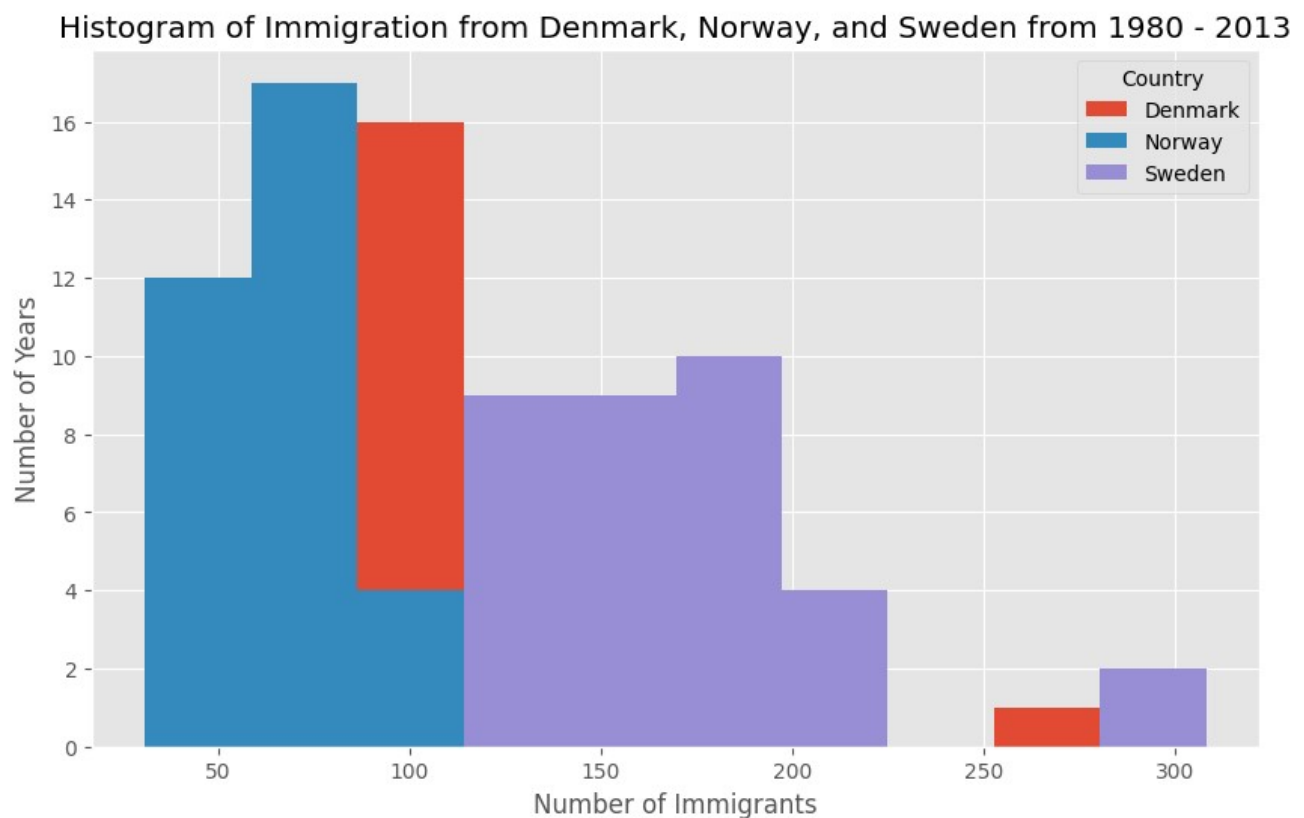
```
df_t = df_can.loc[['Denmark', 'Norway', 'Sweden'], years].transpose()  
df_t.head()
```

Country	Denmark	Norway	Sweden
1980	272	116	281
1981	293	77	308
1982	299	106	222
1983	106	51	176
1984	93	31	128

Now plot:

```
df_t.plot(kind='hist', figsize=(10, 6))
```

```
plt.title('Histogram of Immigration from Denmark, Norway, and Sweden from 1980 - 2013')
plt.ylabel('Number of Years')
plt.xlabel('Number of Immigrants')
plt.show()
```



Result: A histogram with three overlapping distributions. Now each bar represents how many years a country's immigration fell within a given range.

Let's refine it - increase bins to 15, adjust transparency, and use custom colors:

```
count, bin_edges = np.histogram(df_t, 15)
```

```
df_t.plot(kind='hist',
          figsize=(10, 6),
          bins=15,
          alpha=0.6,
```

```

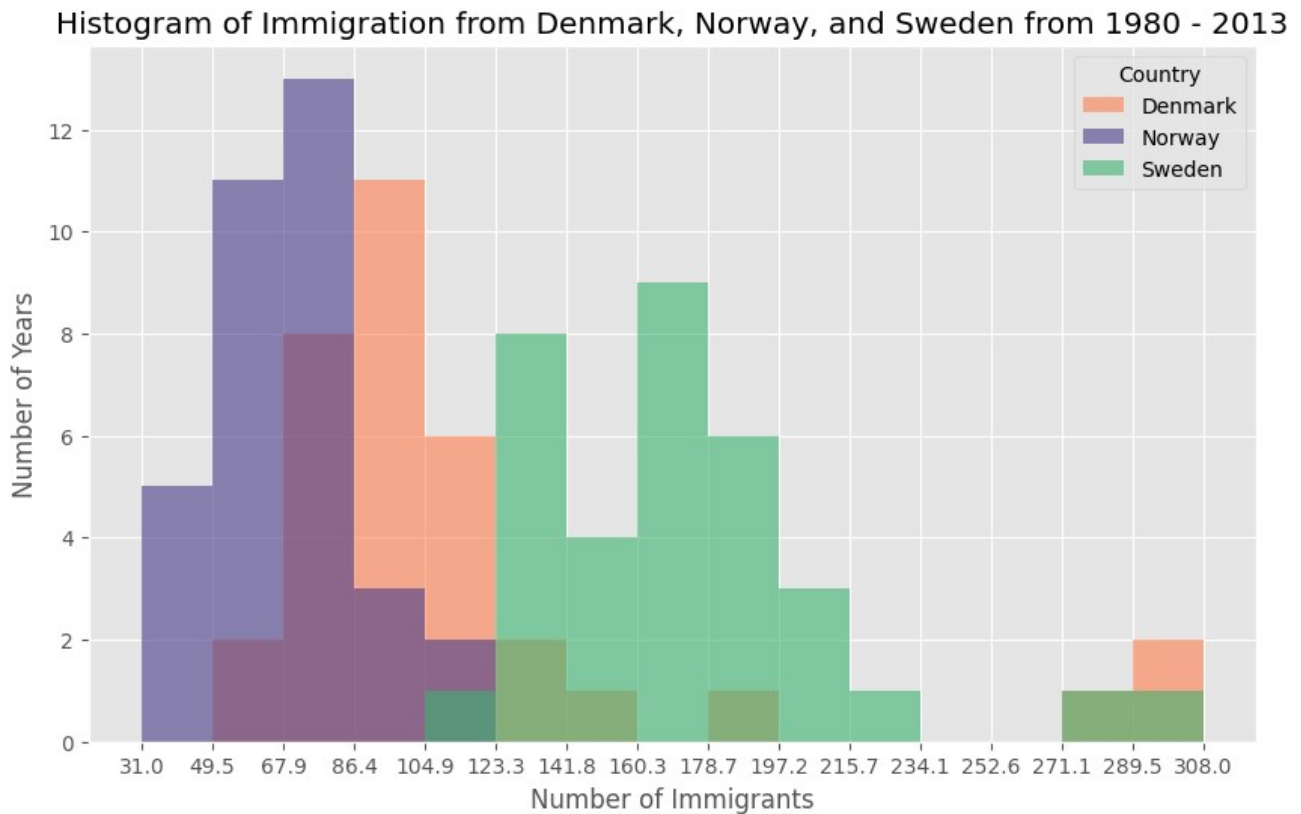
xticks=bin_edges,
color=['coral', 'darkslateblue', 'mediumseagreen']
)

```

```

plt.title('Histogram of Immigration from Denmark, Norway, and Sweden from 1980 -
2013')
plt.ylabel('Number of Years')
plt.xlabel('Number of Immigrants')
plt.show()

```



Result: A cleaner overlapping histogram (**unstacked**) with 15 bins and distinct colors per country.

I can also **stack** them to avoid overlap:

```

count, bin_edges = np.histogram(df_t, 15)
xmin = bin_edges[0] - 10 # first bin value is 31.0, adding buffer of 10 for
aesthetic purposes
xmax = bin_edges[-1] + 10 # last bin value is 308.0, adding buffer of 10 for
aesthetic purposes

```

```

df_t.plot(kind='hist',
          figsize=(10, 6),
          bins=15,
          alpha=0.6,
          xticks=bin_edges,
          color=['coral', 'darkslateblue', 'mediumseagreen'],
          stacked=True,
          xlim=(xmin, xmax) #trim x-axis limits
)

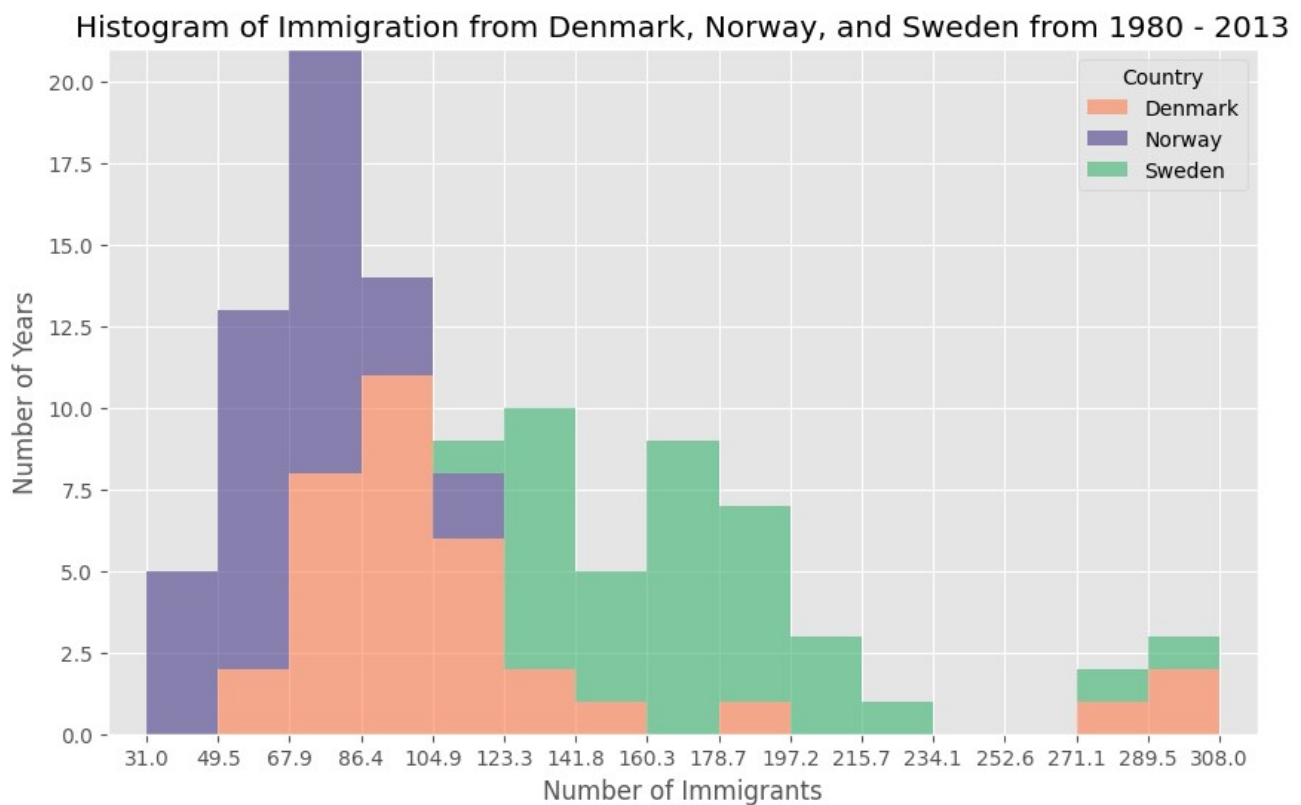
```

```

plt.title('Histogram of Immigration from Denmark, Norway, and Sweden from 1980 -
2013')

```

```
plt.ylabel('Number of Years')
plt.xlabel('Number of Immigrants')
plt.show()
```



Result: A **stacked** histogram where the bars are layered on top of each other rather than overlapping. The x-axis limits are also trimmed to remove excess whitespace.

Tip: Check full listing of colors available:

```
import matplotlib
for name, hex in matplotlib.colors.cnames.items():
    print(name, hex)
```

Exercise: Immigration Distribution for Greece, Albania, and Bulgaria (Scripting Layer)

Task: Use the scripting layer to display the immigration distribution for Greece, Albania, and Bulgaria from 1980 to 2013. Use 15 bins and alpha = 0.35.

```
df_cof = df_can.loc[['Greece', 'Albania', 'Bulgaria'], years]
```

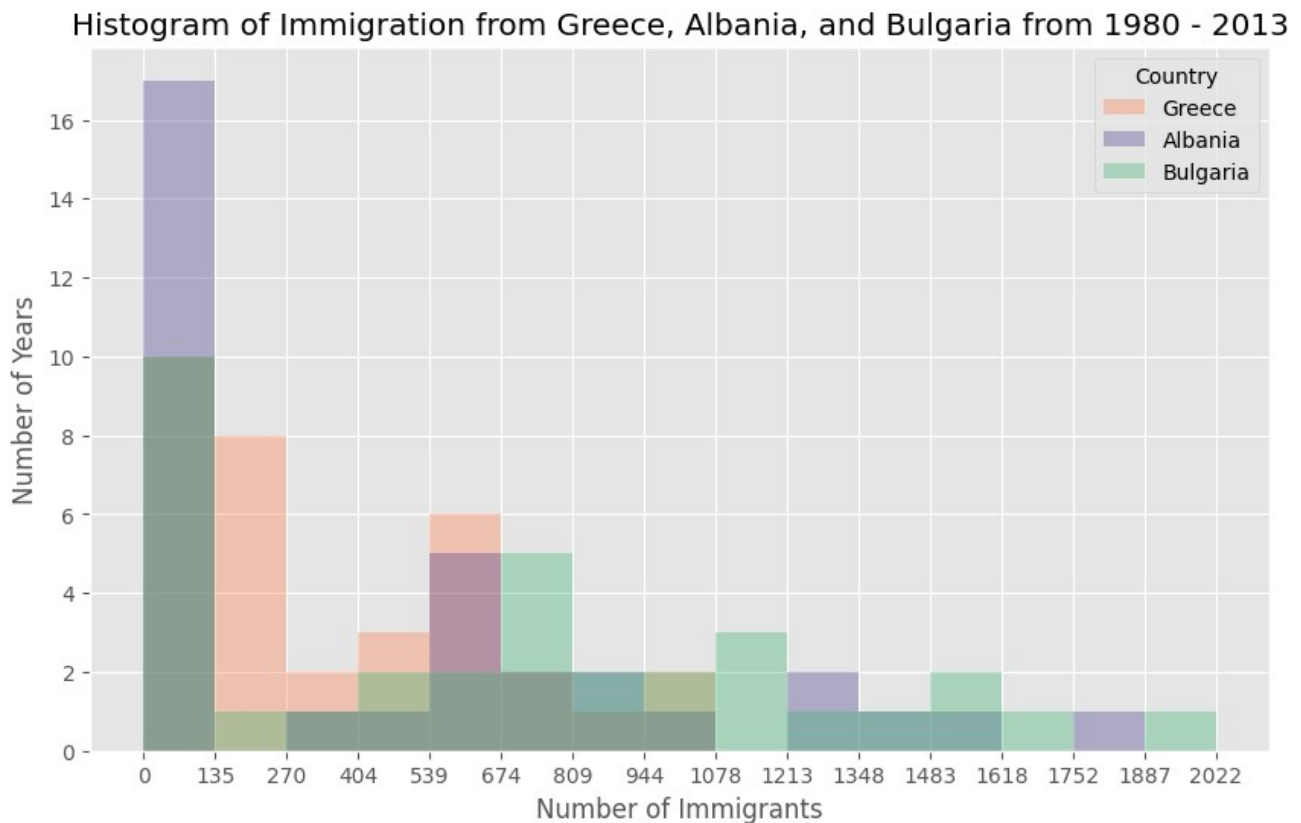
```
df_cof = df_cof.transpose()
```

```
count, bin_edges = np.histogram(df_cof, 15)
```

```
df_cof.plot(kind='hist',
            figsize=(10, 6),
            bins=15,
            alpha=0.35,
            xticks=bin_edges,
            color=['coral', 'darkslateblue', 'mediumseagreen'])
```

)

```
plt.title('Histogram of Immigration from Greece, Albania, and Bulgaria from 1980  
- 2013')  
plt.ylabel('Number of Years')  
plt.xlabel('Number of Immigrants')  
plt.show()
```



Result: An overlapping histogram for three Balkan countries. The distributions show how many years each country's immigration level fell into each bin range.

6. Bar Charts

Bar charts represent the magnitude of a variable (**numerically and categorically**) with the **length** of the bars. I'll explore both **vertical** `kind='bar'` and **horizontal** `kind='barh'` bar charts.

Vertical Bar Plot - Iceland's Immigration Over Time

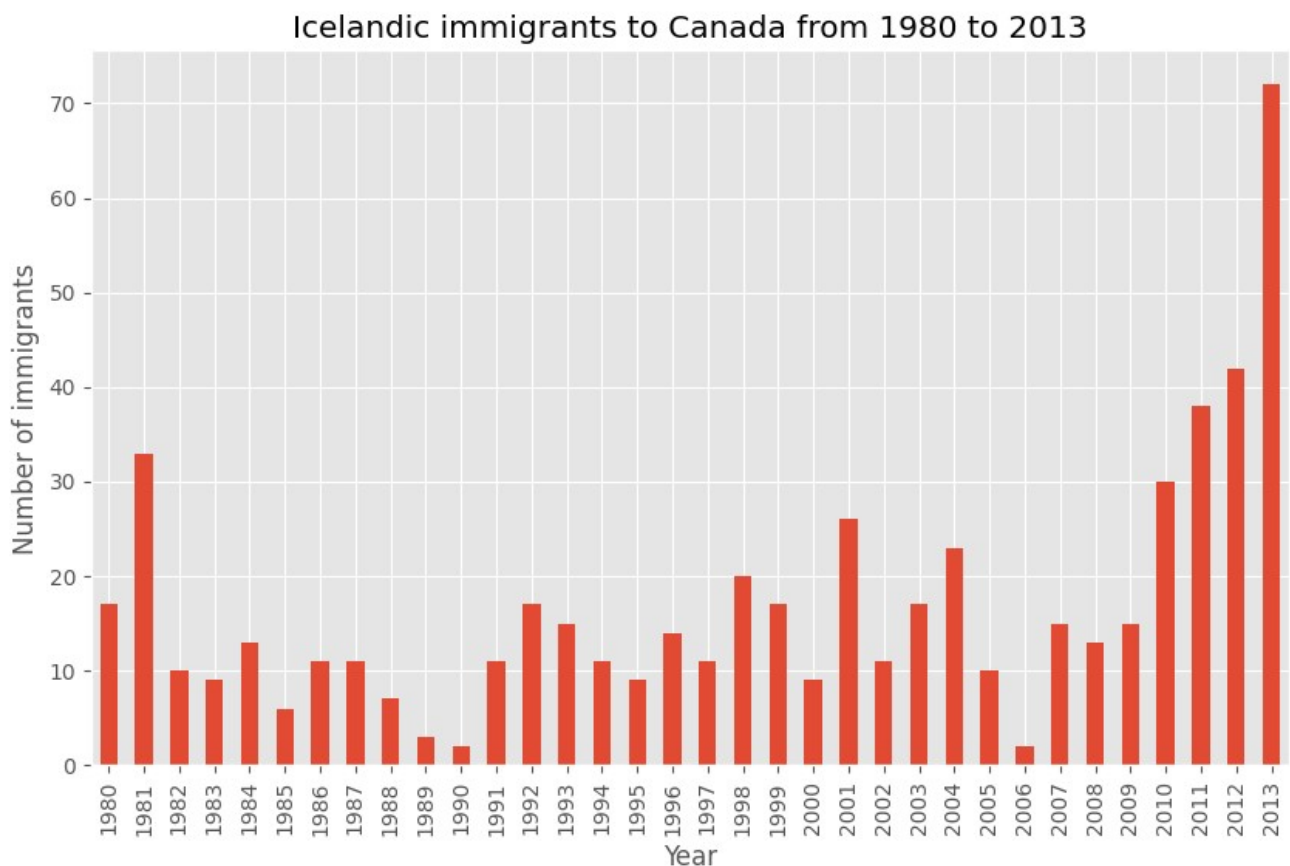
Let's look at Icelandic immigration to Canada. The 2008-2011 financial crisis had a major impact on Iceland's economy, so we might expect to see a change in emigration patterns.

```
df_iceland = df_can.loc['Iceland', years]  
df_iceland.head()
```

```
1980    17
1981    33
1982    10
1983     9
1984    13
Name: Iceland, dtype: int64
```

```
df_iceland.plot(kind='bar', figsize=(10, 6))
```

```
plt.xlabel('Year')
plt.ylabel('Number of immigrants')
plt.title('Icelandic immigrants to Canada from 1980 to 2013')
plt.show()
```



Result: A vertical bar chart showing Icelandic immigration year by year. A noticeable upward trend starts around 2008, coinciding with the financial crisis.

Let's add an **annotation to highlight** this:

for more details on annotations: https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.annotate.html.

```
df_iceland.plot(kind='bar', figsize=(10, 6), rot=90) # rotate the xticks(Labelled points on x-axis) by 90 degrees
```

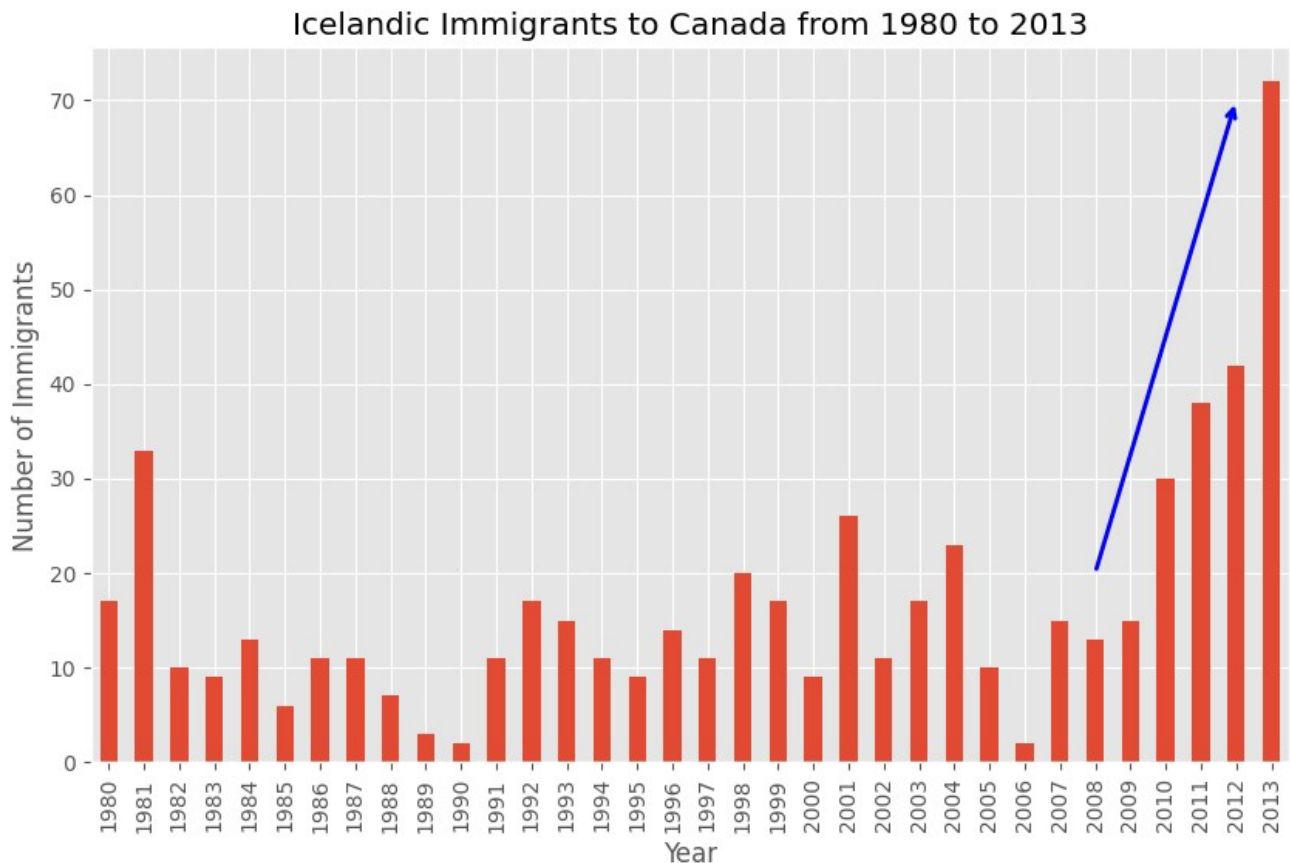
```
plt.xlabel('Year')
plt.ylabel('Number of Immigrants')
plt.title('Icelandic Immigrants to Canada from 1980 to 2013')
```

```
plt.annotate('', # s: str. Will leave it blank for no text
             xy=(32, 70), # place head of the arrow at point (year 2012 , pop 70)
```

```

        xytext=(28, 20), # place base of the arrow at point (year 2008 , pop
20)
        xycoords='data', # will use the coordinate system of the object
being annotated
        arrowprops=dict(arrowstyle='->', connectionstyle='arc3',
color='blue', lw=2) # specifies connection style `arc3` straight line, color of
arrow, and line width
    )
plt.show()

```



Result: Same bar chart with a blue arrow pointing from 2008 to 2012, highlighting the surge.

Let's add **explanatory text** alongside the arrow:

```

df_iceland.plot(kind='bar', figsize=(10, 6), rot=90)

plt.xlabel('Year')
plt.ylabel('Number of Immigrants')
plt.title('Icelandic Immigrants to Canada from 1980 to 2013')

plt.annotate('',
            xy=(32, 70),
            xytext=(28, 20),
            xycoords='data',
            arrowprops=dict(arrowstyle='->', connectionstyle='arc3',
color='blue', lw=2)
            )

plt.annotate('2008 - 2011 Financial Crisis',
            xy=(28, 30), # start the text at at point (year 2008 , pop 30)

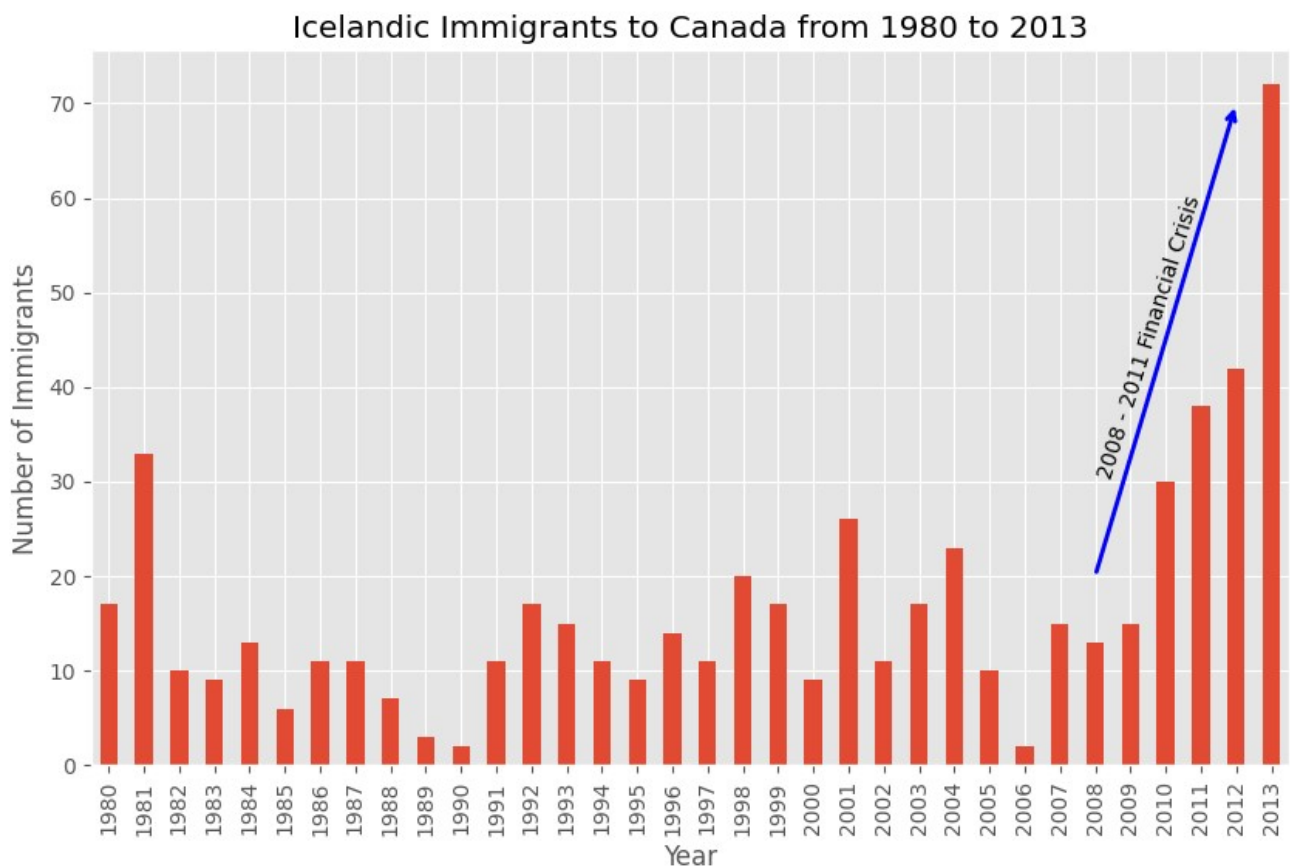
```



```

rotation=72.5, # in degrees (counter clockwise) try&error
va='bottom', # 'center' | 'top' | 'bottom' | 'baseline'
ha='left', # 'center' | 'right' | 'left'
)
plt.show()

```



Result: An arrow and a text label explaining the 2008-2011 financial crisis. Annotations can tell a story within a visualization.

Exercise: Horizontal Bar Plot - Top 15 Countries

Task: Create a horizontal bar plot showing the total number of immigrants to Canada from the **top 15 countries** (1980–2013). Label each bar with the total count.

Step 1: Get the top 15 countries by total immigration.

```
df_can.sort_values(by='Total', ascending=True, inplace=True)
```

```
df_top15 = df_can['Total'].tail(15)
df_top15
```

```

Country
Romania      93585
Viet Nam     97146
Jamaica     106431
France      109091
Lebanon     115359
Name: Total, dtype: int64

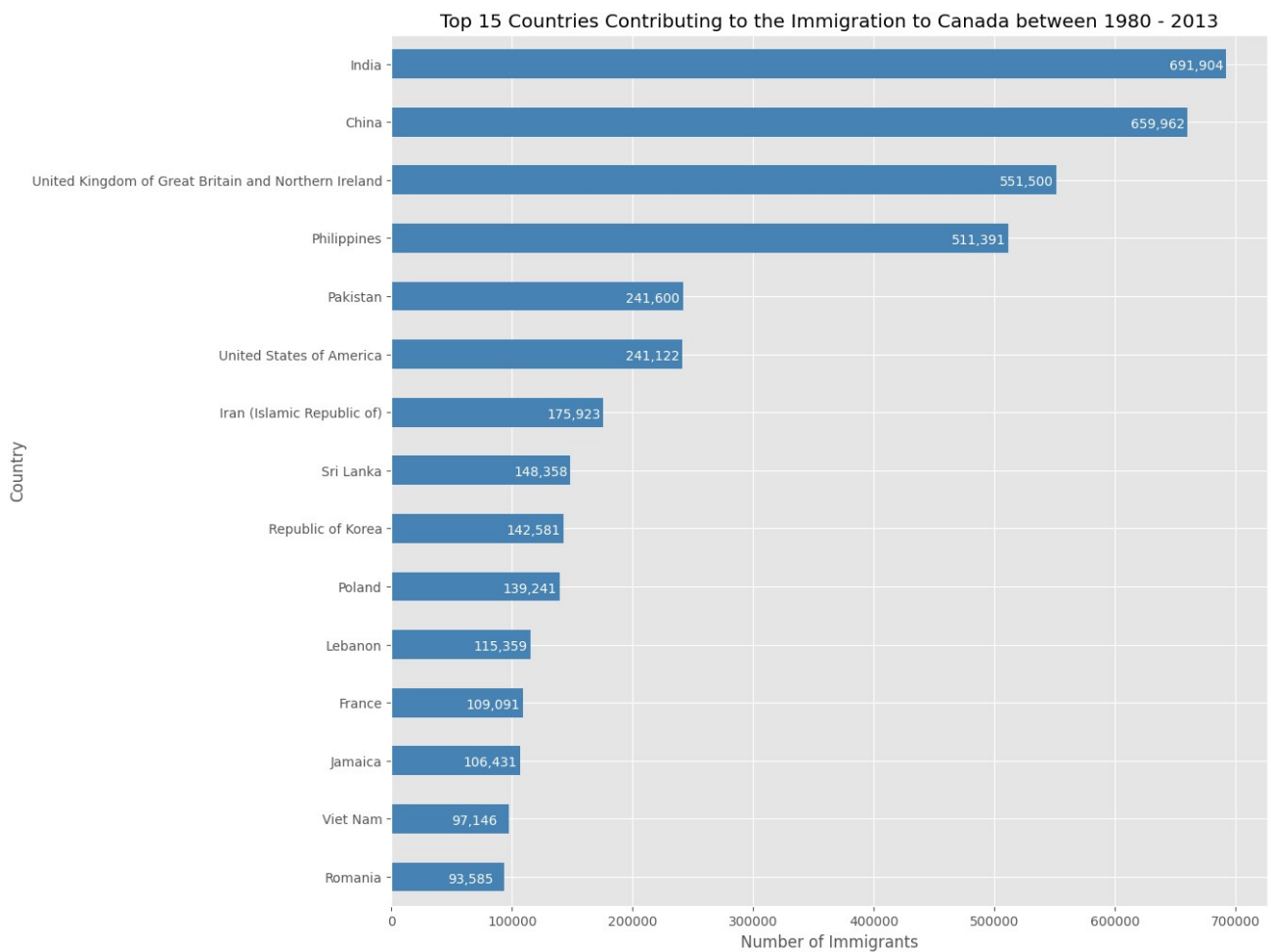
```

Step 2: Plot as a horizontal bar chart with annotations.

```
df_top15.plot(kind='barh', figsize=(12, 12), color='steelblue')
plt.xlabel('Number of Immigrants')
plt.title('Top 15 Countries Contributing to the Immigration to Canada between
1980 - 2013')

for index, value in enumerate(df_top15):
    label = format(int(value), ',') # format int with commas

    # place text at the end of bar (subtracting 47000 from x, and 0.1 from y to
    # make it fit within the bar)
    plt.annotate(label, xy=(value - 47000, index - 0.10), color='white')
plt.show()
```



Result: A horizontal bar chart with white annotation showing the exact immigrant count.

Why horizontal bars? They leave more room for country labels on the y-axis, making them ideal for displaying many categories without overlapping text.

That's the full walkthrough. The dataset is cleaned, indexed by country, and visualized with area plots, histograms, and bar charts.